

A Reproducible Evaluation Framework for Adaptive Fee Policies in Uniswap v4

Abstract—Uniswap v4 allows liquidity providers to adjust swap fees through programmable hooks. However, comparing alternative fee policies requires more than implementing their fee rules: each policy must be deployed under compatible hook permissions, initialized with the same pool state, exposed to identical market conditions, and evaluated using consistent metrics. This paper presents a reproducible framework that integrates dynamic-fee hook implementations, permission-aware deployment, historical market replay, local EVM execution, and comparative analysis. We evaluate five dynamic policies and four static-fee configurations in 29,160 experimental cells covering three token pairs, 72 one-day windows per pair, three gas-price scenarios, and five ordinary-user demand levels. Each policy is paired with a predeclared static 30 bps baseline under identical market conditions and random inputs. The results show that no dynamic policy provides an unconditional advantage over the baseline. Some policies improve liquidity-provider outcomes during high-volatility, low-gas conditions, but the best-performing policy depends on the balance between ordinary-user and arbitrage volume. Higher gas prices eliminate these gains. The findings indicate that dynamic fees are conditionally, rather than universally, beneficial and demonstrate the importance of evaluating fee policies jointly with market conditions, order-flow composition, and implementation cost.

Index Terms—automated market makers, decentralized exchanges, dynamic fees, Uniswap v4, smart-contract hooks, liquidity provision

I. INTRODUCTION

Decentralized exchanges (DEXs) allow users to trade digital assets without a custodial order-matching intermediary. Many DEXs use automated market makers (AMMs), which execute swaps against liquidity provided by market participants according to a deterministic pricing function [1], [2]. Uniswap popularized the constant-product AMM and later introduced concentrated liquidity, allowing liquidity providers (LPs) to allocate capital within selected price ranges [6], [7]. Providing liquidity exposes LPs to changes in both the composition and market value of their position. When an external market price changes before the AMM price, arbitrageurs can trade against the stale pool until the remaining price difference no longer covers the pool fee and transaction cost. Impermanent loss compares the resulting LP position with simply holding the originally deposited assets, whereas loss-versus-rebalancing measures the cost of passive rebalancing at stale prices relative to continuous rebalancing at the external price [3]. Concentrated liquidity introduces an additional trade-off: a narrow range earns a larger share of fees while active but stops earning when the market price leaves that range [8].

Swap fees compensate LPs for supplying liquidity and accepting these risks [9]. However, a single static fee must serve

very different market conditions. A high fee may discourage ordinary trading during calm periods, whereas a low fee may undercompensate LPs when volatility and arbitrage activity increase. Dynamic-fee policies attempt to address this trade-off by adjusting the fee using signals such as price movement, volatility, trade direction, or trade size. Their benefit is not automatic: a more responsive fee may increase revenue from risky flow, but it may also reduce executed volume and consume additional gas. The central question is therefore whether, and under which conditions, a dynamic policy produces a better LP outcome than a well-chosen static fee. In this study, *dynamic* refers only to the swap fee; the hook does not directly set the asset price, which remains determined by the pool state, available liquidity, and executed swaps.

Prior research has examined dynamic fee setting from theoretical and simulation-based perspectives. Fritsch studies optimal fees for constant-function market makers [10]. Lebedeva *et al.* compare block-adaptive, trade-adaptive, and oracle-informed asymmetric policies [4]; Baggiani *et al.* formulate fee selection as a stochastic-control problem [5]; and Campbell *et al.* study fee setting when an AMM competes with a centralized exchange [11]. Agent-based work further examines heterogeneous traders, latency, arbitrage, and MEV searchers [12]. These studies provide economic models and candidate policies, but they do not provide a common Uniswap v4 implementation and execution path through which several policies can be deployed and compared under identical conditions.

Uniswap v4 provides the mechanism needed to implement these policies. A hook is an external smart contract attached to a pool at initialization and invoked at specified points in the pool lifecycle. For a dynamic-fee pool, the hook can update the stored LP fee or return a fee override for an individual swap [13], [14]. Implementing a policy as a usable hook nevertheless requires more than translating its fee equation into Solidity. The implementation must define callback permissions, deploy the hook at a permission-compatible address, initialize the required Uniswap v4 contracts and pool, provide liquidity and price inputs, execute swaps, and collect comparable outputs. If this surrounding infrastructure is implemented separately for every policy, differences in deployment or simulation code may be mistaken for differences caused by the fee policy itself.

To address this gap, we develop a reproducible implementation and evaluation framework for Uniswap v4 dynamic-fee hooks. The framework provides five Solidity hook implementations, permission-aware Foundry deployment scripts, local pool initialization, Chainlink-compatible mock price feeds, and a historical replay and analysis pipeline. All fee

TABLE I
POSITIONING RELATIVE TO REPRESENTATIVE AMM AND DYNAMIC-FEE RESEARCH.

Work	Primary contribution	Relationship to this framework
Angeris <i>et al.</i> [1], [2]	Formal analysis and optimization of constant-function markets	Supplies the AMM foundation, not hook tooling
Milionis <i>et al.</i> [3]	LVR formulation for adverse-selection cost	Supplies the LP-risk benchmark used in the evaluation
Lebedeva <i>et al.</i> [4]	Block-, deal-, and oracle-informed fee policies	Supplies a policy family represented by the implementations
Baggiani <i>et al.</i> [5]	Stochastic-control formulation of dynamic fees	Supplies theoretical fee-design guidance
This work	Common hook implementations, deployment, historical replay, and controlled comparison	Supplies shared Uniswap v4 experimentation infrastructure

policies use the same deployment procedure, starting liquidity, market inputs, trader model, and result-processing code. Consequently, the experiment isolates the fee policy as the principal difference between matched runs.

We evaluate five dynamic policies against four static-fee configurations using 2024 Binance-derived price and volume data for ETH/SHIB, ETH/USDC, and USDC/USDT. For each pair, 72 one-day windows are divided equally among low-, medium-, and high-volatility regimes. Every policy is tested under three gas-price scenarios and five levels of potential ordinary-user demand, producing 29,160 experimental cells. The simulation includes ordinary users and a gas-aware arbitrageur. Each policy result is paired with a static 30 bps run using the same token pair, historical window, gas price, demand level, starting liquidity, and random inputs.

No evaluated dynamic policy outperforms the static 30 bps baseline across all conditions. Statistically significant improvements appear only in particular high-volatility, low-gas settings, and the best-performing policy changes with the balance between ordinary-user and arbitrage volume. As gas prices increase, the additional execution cost of the dynamic hooks removes these gains. The results therefore show that dynamic fees are conditionally, rather than universally, beneficial.

The contributions of this work are:

- A common Uniswap v4 implementation and deployment framework for five dynamic-fee policies, including shared interfaces, permission-aware hook deployment, pool initialization, mock price feeds, and contract-level gas measurement.
- A reproducible historical-replay methodology that compares dynamic and static fee policies under matched market windows, gas prices, demand levels, starting liquidity, and random user-flow inputs.
- A 29,160-cell empirical evaluation showing that dynamic-fee hooks do not outperform a static 30 bps baseline unconditionally, but can improve LP outcomes in specific high-volatility, low-gas settings.

The evaluation uses local EVM execution, historical market observations, and modeled trader behavior. It does not reproduce live oracle operation, mempool competition, block-builder ordering, strategic routing, or endogenous public-chain liquidity. The results should therefore be interpreted as controlled comparative evidence, not as predictions of live-pool profitability. The objective is to determine how the evaluated

fee policies behave when market conditions, order-flow composition, and implementation cost are varied systematically.

II. BACKGROUND AND RELATED WORK

A. AMM Fees and LP Outcomes

For reserves x and y , a constant-product AMM maintains $xy = k$. If a trader supplies Δx and pays proportional fee f , the simplified output is

$$\Delta y = y - \frac{xy}{x + (1 - f)\Delta x}. \quad (1)$$

A higher fee can increase LP revenue per executed swap, but it also worsens trader execution and may reduce volume. Arbitrage links the pool price to external markets: when the price difference covers the pool fee and transaction cost, arbitrageurs trade against the pool and move its price toward the external reference [1], [2], [15].

Let $V_{LP}(T)$ denote the terminal value of the LP's token holdings excluding separately recorded fees, $V_H(T)$ the terminal value of holding the originally deposited assets, and $R_f(T)$ accumulated fee income. We define

$$IL(T) = V_H(T) - V_{LP}(T), \quad (2)$$

$$LPNet(T) = R_f(T) - IL(T). \quad (3)$$

A positive $LPNet(T)$ means that liquidity provision outperformed holding; a negative value means that holding produced the better outcome. A dynamic fee may therefore improve the LP outcome through additional revenue without eliminating the stale-price mechanism that creates adverse selection. LVR isolates this mechanism by comparing the AMM with continuous rebalancing at the external reference price [3]. Related work has also examined when fee accumulation can offset impermanent loss [16].

A static pool applies $f_t = f$, whereas a dynamic policy computes

$$f_t = \mathcal{F}(s_t, z_t; \theta), \quad (4)$$

where s_t is observable on-chain state, z_t is optional external information, and θ is the policy parameter vector. Directional policies may return separate fees $f_t^{0 \rightarrow 1}$ and $f_t^{1 \rightarrow 0}$.

B. Dynamic-Fee Research

Dynamic-fee policies use signals such as volatility, inventory, pool-to-oracle price deviation, trade direction, or trade size. Lebedeva *et al.* compare block-level, transaction-level, and oracle-informed mechanisms [4]. Baggiani *et al.* study the balance between arbitrage deterrence and uninformed flow [5], while Campbell *et al.* relate optimal fees to volatility and competing-venue costs [11]. Agent-based research suggests that dynamic fees may improve LP profitability by increasing compensation during toxic-flow states rather than by eliminating LVR [12].

These studies provide economic models and candidate policies. Our contribution is complementary: a shared Uniswap v4 implementation, deployment, historical-replay, and analysis path for comparing several policy families under identical conditions. Table I summarizes this positioning.

C. Uniswap v4 and External Price Data

Uniswap v4 manages pools through a singleton `PoolManager` and uses flash accounting to settle intermediate balance changes [13]. Hooks are external contracts invoked at selected points in a pool’s lifecycle. Their callback permissions are encoded in the low-order bits of the hook address, so deployment must produce an address that matches the declared permissions [17], [18]. A dynamic-fee hook can update the stored fee through `updateDynamicLPFee` or return a per-swap override from `beforeSwap` [14]. OpenZeppelin provides reusable abstractions for callback validation and fee management [19].

Oracle-dependent policies also require an external price reference. For two USD-denominated feeds, the external ratio is

$$r_t^{\text{ext}} = \frac{p_{0,t}}{p_{1,t}}. \quad (5)$$

Production implementations must normalize feed decimals and reject invalid or stale observations. In the evaluation, Chainlink-compatible mock feeds reproduce historical prices and provide identical deterministic inputs to matched policy runs [20]. They validate contract integration but do not reproduce live-oracle aggregation, latency, or failure behavior.

III. FRAMEWORK DESIGN

A. Architecture and Workflow

The framework provides a common execution environment for implementing, deploying, and evaluating dynamic-fee policies in Uniswap v4. It consists of three layers. The *contract layer* contains the Solidity hook implementations, shared fee interface, and policy-specific state. The *deployment layer* uses Foundry scripts to deploy the required Uniswap v4 contracts, tokens, mock price feeds, hook, and liquidity pool. The *experiment layer* replays historical market observations, executes simulated user and arbitrage swaps, and converts the resulting contract and pool outputs into comparative metrics.

The process proceeds as follows. Each experiment selects a policy implementation and parameter set, initializes a local

Uniswap v4 pool with a common starting price and liquidity position, and deploys the required mock feeds. Historical observations update the external price reference, after which simulated swaps invoke the deployed hook. The experiment records the applied fees, token balances, gas usage, executed volume, and LP outcome.

All policies reuse the same deployment procedure, pool initialization, trader model, historical inputs, and result-processing pipeline. This separation allows matched runs to differ principally in their fee policy rather than in independently developed supporting infrastructure. The framework produces deployment logs, transaction traces, contract-level gas measurements, and structured experiment outputs for subsequent analysis.

B. Hook Execution and Deployment

The five dynamic implementations share a fee-oriented interface based on `BaseOverrideFee`. When a swap invokes a hook, the common execution path first verifies that the callback originated from the designated `PoolManager` and determines the swap direction. The hook then reads its policy state and, when required, an external price observation. It computes the applicable fee, enforces the configured bounds, and returns the fee in the format expected by Uniswap v4. After the swap, the hook updates only the state required by its policy. Some policies update after every executed trade, whereas others update at most once per block.

The evaluated implementations are `BAHook`, `DAHook`, `ABHook`, `VolatilityHook`, and `MEVChargeHook`. They respond, respectively, to signals derived from external price direction, executed trade direction, endogenous pool-price movement, recent external-price volatility, or trade size and sender activity. Section IV defines their update rules and experimental parameters.

Uniswap v4 encodes hook permissions in the low-order bits of the hook address. Deployment must therefore find a compatible `CREATE2` salt, or use an equivalent deterministic procedure, to obtain an address with the required permission pattern [18]. The deployment scripts initialize the `PoolManager`, routing and position-management contracts, mock tokens, price feeds, hook, and pool. They then set the initial pool price and provide the common starting liquidity. Before a pool is used, the framework verifies that the hook address matches its declared callbacks because an incompatible permission bitmap can prevent the required hook logic from executing.

Binance observations and Chainlink-compatible mock feeds serve different purposes. Binance supplies the offline price and volume series used by the historical replay. The mock feeds expose the relevant observations through the interface expected by oracle-dependent hooks. This arrangement provides identical, deterministic price inputs across matched runs. It tests contract integration but does not reproduce live-feed aggregation, update latency, or failure behavior.

C. Testing and Reproducibility

Testing separates shared infrastructure behavior from policy-specific fee logic. Common Foundry tests verify hook deployment, address permissions, pool initialization, callback authorization, liquidity provision, and successful swap execution. Policy-specific tests verify fee direction, update frequency, parameter bounds, and state transitions. Historical replay tests execute the complete market sequence and collect fee, balance, gas, volume, and LP-outcome data.

A reproducible experimental result must identify both the evaluated policy and its execution context. We represent each experimental cell as

$$E = \langle h(C), \theta, D, P_0, L_0, W, S \rangle, \quad (6)$$

where $h(C)$ identifies the hook implementation, θ is its parameter vector, and D identifies the compiler and dependency versions. The initial pool price and liquidity are denoted by P_0 and L_0 , respectively; W identifies the historical market window; and S contains the gas-price scenario, ordinary-user demand level, and random seed. A fingerprint of the historical candle data additionally identifies the replay input.

All swaps execute against local Uniswap v4 contracts in Foundry rather than being calculated exclusively by an off-chain model. Gas usage is measured from these executions, while LP holdings are reconstructed from pool state and token balance changes. Reusing the same market window, starting state, gas scenario, and random inputs across policies enables paired comparisons in which the surrounding experimental conditions remain fixed.

IV. EXPERIMENTAL EVALUATION

The framework’s evaluation layer runs every fee policy and a family of static baselines over identical replayed market histories and scores the liquidity provider (LP) against simply holding the deposited tokens. The experiment evaluates the complete deploy–execute–measure path at scale and provides a controlled economic comparison of the implemented dynamic-fee policies, including their contract-level gas overhead. **Artifact availability:** *The source code, deployment and experiment scripts, policy configurations, and data-processing utilities will be made publicly available on GitHub upon acceptance.*

A. Evaluated Fee Policies

All policies share the fee box $[1, 100]$ bps, and every updated fee is clamped to that box. Directional fees are written $f^{0 \rightarrow 1}$ and $f^{1 \rightarrow 0}$; every fee starts at 30 bps unless noted. Gas is the EVM-measured median per swap, including the 21k intrinsic transaction cost. Table II summarizes each policy’s signal, update rule, and concrete parameters.

DAHook, ABHook, and MEVChargeHook are endogenous stands their oracle arguments are unused; BAHook and VolatilityHook read two Chainlink-compatible feeds, which is what their higher gas reflects.

B. Simulated Market and Reproducibility

One *cell* of the experiment’s factorial matrix, a combination of fee policy, trading pair, day-window, and gas-price scenario, replays a *window* of 1,440 one-minute Binance candles from 2024 against a Uniswap v4 pool holding a 20M USDT basket, split equally by value into a full-range position. The pool opens at the external market price of the first candle; a mock Chainlink-compatible feed tracks each candle’s close; the first 60 candles execute but are excluded from measurement. Three pairs are studied: ETH/SHIB and ETH/USDC (volatile) and USDC/USDT (stable). Windows are stratified by realized volatility into terciles (24 calm, 24 medium, and 24 stormy days per pair), so market regimes are represented by design.

Every cell is identified by the manifest of Eq. (6), so any change to a contract or input identifies exactly the affected cells. Because every swap executes in a real EVM (Forge), gas is *measured* rather than assumed: a calibration run records each policy’s median gas per swap (76k for the static hook, 88–124k for the adaptive ones), and these measured figures feed back into both traders’ decisions below. Fees follow v4 accounting exactly: they accrue to the position and do not compound.

C. Simulated User and Arbitrage Flow

Following the standard microstructure framing, order flow is split into informed arbitrageurs and uninformed users (UUs).

1) *Arbitrageur:* With fee fraction f and $\gamma = 1 - f$, the pool price may sit anywhere inside the no-arbitrage band $[P\gamma, P/\gamma]$ around the external price P . Once per candle, after the retail flow, the arbitrageur moves it to the nearest band edge if profitable *after gas*: with pool liquidity L , sqrt-price s and target t , the closed-form profit

$$\Pi_{0 \rightarrow 1}^* = \frac{L(s - t)^2}{s}, \quad \Pi_{1 \rightarrow 0}^* = \frac{L(t - s)^2}{\gamma s}, \quad (7)$$

must exceed $g p_{\text{gas}}$, where g is the policy’s measured median gas and p_{gas} the scenario’s gas price (5, 20, or 80 gwei), valued at the ETH price series. For the size-dependent MEVChargeHook the optimum is found by ternary search because profit is unimodal in size.

2) *Uninformed users:* Ordinary-user behavior follows the acceptance model proposed in [4]: a candidate trade evaluates the normalized outcome ratio

$$r = \frac{V_{\text{out}} - V_{\text{in}} - G}{V_{\text{out}} + V_{\text{in}} + G}, \quad (8)$$

where V_{in} and V_{out} are the two legs valued at external prices, and G is the trade’s own gas. Execution has a probability

$$P_{\text{exec}} = \begin{cases} 1, & r \geq 0, \\ e^{-\lambda|r|}, & r < 0, \end{cases} \quad (9)$$

with parameter $\lambda = \ln 2 / |r(30 \text{ bps})| \approx 461.4$, so that half of the willing flow declines at a 30 bps total transaction cost.

Each minute c , each direction independently draws one candidate trade of size

$$q(c) = \frac{v_0(c)}{2} e^{\sigma Z - \sigma^2/2}, \quad Z \sim \mathcal{N}(0, 1), \quad \sigma = 1, \quad (10)$$

TABLE II
THE EVALUATED FEE POLICIES.

Policy	Update	Rule	Parameters	Gas Used
Static	never	constant fee, both directions	$f \in \{5, 30, 60, 100\}$ bps; 30 bps = baseline	76–77k
DAH _{hook}	every trade	after a swap in direction d : $f^d \leftarrow f^d + \delta$, $f^{-d} \leftarrow f^{-d} - \delta$	$\delta = 1$ bps	88k
BAH _{hook}	once per block	external ratio $\rho_b = p_{0,b}/p_{1,b}$ from two feeds: $f^{0 \rightarrow 1} \leftarrow f^{0 \rightarrow 1} + \delta \operatorname{sgn}(\rho_{b-1} - \rho_b)$, opposite side mirrored	$\delta = 5$ bps	108k
ABH _{hook}	every trade	Δ_t = relative $\sqrt{\text{price}}$ move of the <i>pool itself</i> : $f^{0 \rightarrow 1} \leftarrow f^{0 \rightarrow 1} - c \Delta_t$, $f^{1 \rightarrow 0} \leftarrow K - f^{0 \rightarrow 1}$	$c = 100$ bps per unit move; $K = 60$ bps (each side in $[1, 59]$ bps)	94k
VolatilityHook	once per block	$f = f_{\text{base}} + c \hat{\sigma}$, both directions; $\hat{\sigma}$ = EMA per-minute st. dev. of the external ratio's relative change	$f_{\text{base}} = 20$ bps, $c = 2 \times 10^6$, $\alpha = 0.1$	123k
MEVChargeHook	every trade	30 bps base + surcharges in trade size and for repeated senders	cooldown 15 s; range $[30, 100]$ bps	95k

where $v_0(c)$ is the minute's potential demand (Eq. (12) below), split equally between the two directions. The lognormal multiplier is mean-preserving ($\mathbb{E}[e^{\sigma Z - \sigma^2/2}] = 1$), so randomness produces realistic heavy-tailed trade sizes without distorting the calibrated volume; the trade then executes all-or-nothing with probability P_{exec} . Trades are discrete and pre-generated from a recorded seed.

3) *Demand calibration*: The minute-level potential demand $v_0(c)$ combines a measured intraday shape with one assumed scale parameter, κ . With $V_0(c), V_1(c)$ the 2024 per-minute Binance quote volumes of the pair's two legs, the intraday shape is

$$\hat{s}(c) = \frac{g(c)}{\langle \sum_{c' \in \text{day}} g(c') \rangle}, \quad g(c) = \sqrt{V_0(c) V_1(c)}, \quad (11)$$

where the average is taken over the days of 2024, so $\hat{s}$ sums to one over an average day, and

$$v_0(c) = \kappa \cdot \text{basket} \cdot \hat{s}(c), \quad (12)$$

with the basket the pool's initial 20M USDT deposit. Hence κ is the pool's daily turnover relative to its own TVL. We treat $\kappa = 1$ as a reference point. The experiment is repeated at $\kappa \in \{0.03, 0.1, 0.3, 1, 3\}$ which moves the measured arbitrage share of volume across 89%, 58%, 25%, 13%, 16% (Fig. 1).

D. Design and Statistics

One operating point is a full factorial over 9 policy configurations (the five hook policies plus static baselines at 5/30/60/100 bps), times 3 pairs, 72 windows, and 3 gas scenarios: 5,832 cells. Five operating points give 29,160 cells. The LP outcome of a cell is the fee-adjusted net result $\text{LPNet} = R_f - IL$ of Eq. (3): the LP's position and accrued fees against simply holding the deposit, marked at end-of-window prices.

Every comparison is *paired*: a policy's window result is differenced against the 30 bps static baseline on the same window, pair, and gas price, removing the day-to-day variation that dwarfs policy effects. Effects are reported as the median

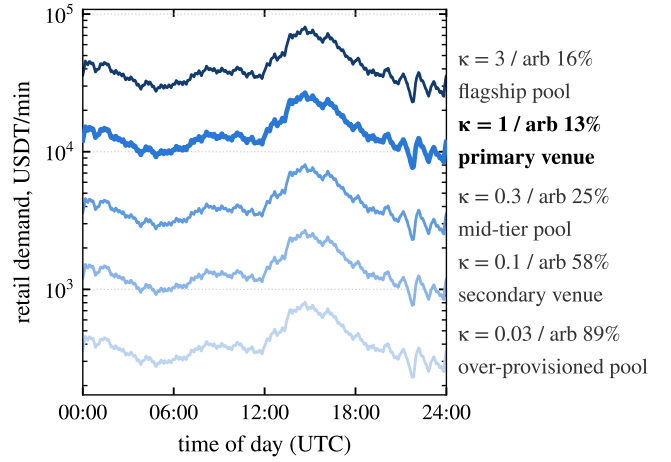


Fig. 1. Demand calibration, Eq. (12): the intraday shape $\hat{s}(c)$ is measured (median 2024 day of the pair's Binance legs); κ scales its height, and the height sets the arbitrage share of volume the experiment measures.

paired difference with a 95% percentile-bootstrap confidence interval over 10,000 seeded resamples. Headline numbers aggregate cell-first (one median per pair×regime×gas cell, then across cells), so three gas scenarios of one day are never treated as independent observations.

E. Results

1) *Aggregate Performance*: Table III reports the aggregate comparison at the reference demand level, $\kappa = 1$. For each matched run, the advantage of policy p over static 30 bps is

$$\Delta_{p,w} = \text{LPNet}_{p,w} - \text{LPNet}_{30,w}, \quad (13)$$

where w identifies the common historical window and experimental conditions. A positive value favors policy p , whereas a negative value favors static 30 bps.

The aggregate estimate covers 18 strata formed by two volatile pairs, three volatility regimes, and three gas-price scenarios. None of the five dynamic hooks has a statistically

TABLE III

PAIRED LP-OUTCOME DIFFERENCE RELATIVE TO STATIC 30 bps AT $\kappa = 1$. VALUES ARE MEDIAN ACROSS THE 18 PAIR $\times$ VOLATILITY $\times$ GAS STRATA WITH 95% BOOTSTRAP CONFIDENCE INTERVALS. POSITIVE VALUES FAVOR THE EVALUATED POLICY.

Policy	Δ vs. 30 bps, USDT/window
Static 60 bps	-2 [-186, +266]
BAHook	-100 [-453, +350]
DAHook	-235 [-381, -104]
ABHook	-460 [-630, -288]
VolatilityHook	-752 [-1069, -280]
MEVChargeHook	-913 [-1659, -728]

significant positive aggregate advantage. BAHook is not distinguishable from static 30 bps, while DAHook, ABHook, VolatilityHook, and MEVChargeHook produce significantly lower LP outcomes.

Repeating the aggregate comparison at the other four demand levels produces the same principal conclusion: no dynamic hook has a statistically significant positive aggregate advantage over static 30 bps at any evaluated κ . Some hooks, however, have statistically significant negative effects.

Decomposing the paired outcomes shows that the impermanent-loss component differs by only single-digit USDT across policies, whereas fee-income differences reach hundreds of USDT. In this simulation, arbitrage returns the pool close to the external price under every policy by the end of each window. Consequently, the evaluated policies affect LP outcomes mainly through the fees collected from executed flow. Their higher gas requirements also affect outcomes indirectly by changing which user and arbitrage trades remain economical. These results apply to the present trader model and should not be interpreted as evidence that dynamic fees cannot affect adverse selection in other market settings.

2) *Conditional performance*: Splitting results by both experimental axes together shows a consistent pattern (Fig. 2): every policy’s advantage over the baseline improves monotonically with volatility and degrades monotonically with gas, in every regime and at every κ . At the reference point ($\kappa=1$), BAHook beats the baseline by +1,556 [+894, +2,501] USDT per window in high-volatility windows at 5 gwei, but loses or ties at 80 gwei. Which policy carries this advantage shifts with the flow mix (Fig. 4): BAHook at retail-dominated κ , VolatilityHook and DAHook at informed-heavy κ .

These conditional findings are stable across random seeds: five seed repetitions using 16 paired windows each preserves the sign of every conditional finding (Fig. 3), and ABHook stays significantly negative at nearly every seed, showing the procedure also detects real disadvantages.

3) *Gas overhead*: The gas gradient follows directly from the policies’ own overhead. Oracle-reading hooks cost 108–124k gas per swap against the static hook’s 76k, and that difference enters retail’s acceptance test through G in Eq. (8). The informed share of volume rises with gas accordingly, from 12.4% to 13.7% across 5→80 gwei at $\kappa=1$, as small retail trades are priced out first.

Overall, none of the evaluated dynamic policies outperforms

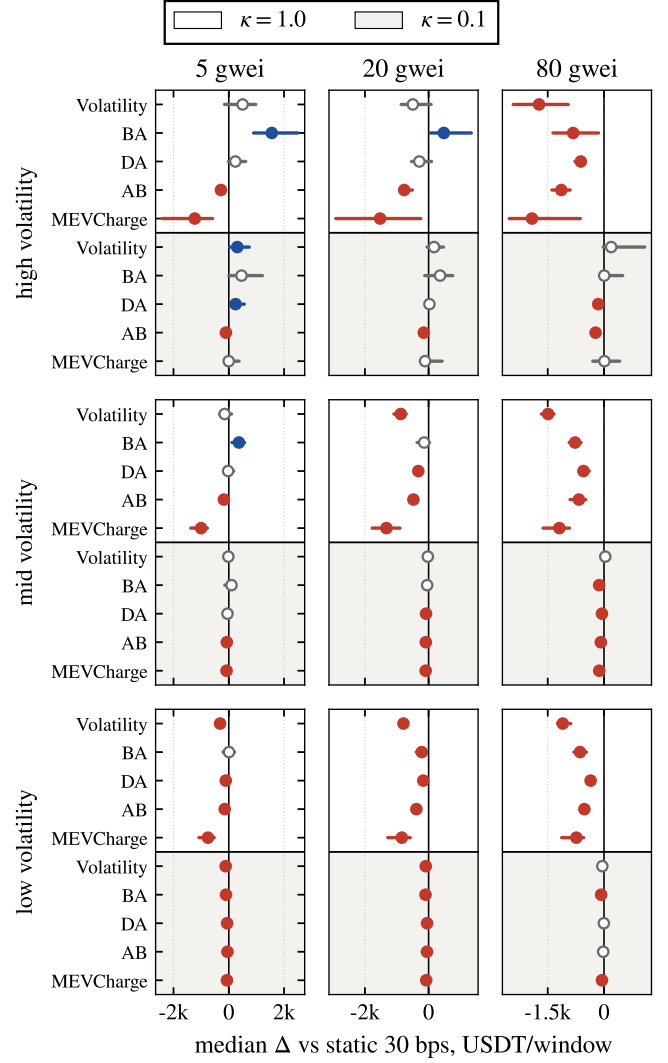


Fig. 2. Paired LP-outcome difference between each dynamic hook and static 30 bps, separated by volatility regime and gas price. Points to the right of zero favor the hook; points to the left favor the baseline. Each panel shows $\kappa = 1$ in the upper white region and $\kappa = 0.1$ in the lower shaded region. Filled markers have 95% bootstrap intervals that exclude zero: blue indicates a significant advantage and red a significant disadvantage. Open grey markers are not statistically distinguishable from the baseline. Each estimate pools 48 matched windows from ETH/SHIB and ETH/USDC.

a well-chosen static fee unconditionally. A reproducible conditional advantage exists in high-volatility windows at low gas prices; which policy carries it depends on the informed share of flow, and the advantage disappears as higher hook gas costs reduce the set of economically executable trades. For the framework, what matters here is that this conditional structure could be measured at all: paired, seed-checked, and decomposed along pre-declared axes, with every policy running through the same implementation path.

V. SECURITY, VALIDITY, AND DISCUSSION

A. Security Considerations

Dynamic-fee hooks execute inside the swap lifecycle and enlarge the trusted code surface. Production implementa-

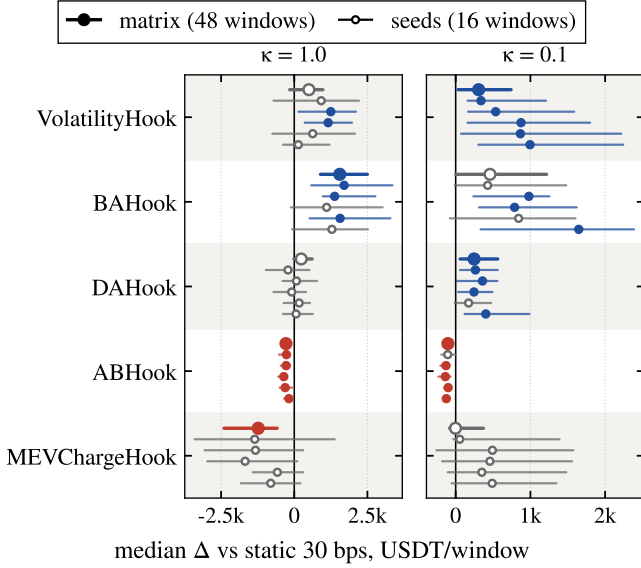


Fig. 3. Robustness of the high-volatility, 5 gwei estimates to the random user-flow seed. For each policy, the bold marker shows the main factorial estimate from 48 paired windows, followed by five seed repetitions using 16 paired windows each. Filled markers have 95% bootstrap intervals that exclude zero. The repetitions preserve the sign of the corresponding main estimate, while their intervals are wider because they use one-third as many windows.

tions should restrict setters, accept callbacks only from the designated `PoolManager`, enforce protocol and policy fee bounds, normalize oracle decimals, reject stale or nonpositive rounds, and handle division and casts safely. Address-permission validation is essential. Uniswap’s security guidance emphasizes that incorrectly encoded permissions or custom deltas can produce unintended execution paths [21].

The historical-data and oracle interface forms a separate trust boundary. The implementation must check feed identity, sign, decimals, and timestamp and define its behavior when observations are unavailable. Contract- and data-boundary threats admit code-level tests, whereas historical-replay bias requires common baselines, multiple regimes, and explicit experimental reporting.

B. Validity and Design Trade-offs

Local EVM execution provides reproducible contract behavior but does not reproduce public-chain market microstructure. The historical replay omits mempool visibility, block-builder ordering, latency, competitive gas bidding, strategic arbitrage, and endogenous routing. The results also depend on the assumed starting liquidity, trade-size distribution, observation frequency, user-acceptance model, and external-price process. Although the evaluation covers three pairs and 72 one-day windows per pair, it does not establish generality across all assets, liquidity ranges, pool depths, or live-market conditions. The findings should therefore be interpreted as controlled comparative evidence rather than predictions of live-pool profitability.

The framework holds the historical window, starting state, gas scenario, demand level, and random inputs constant while

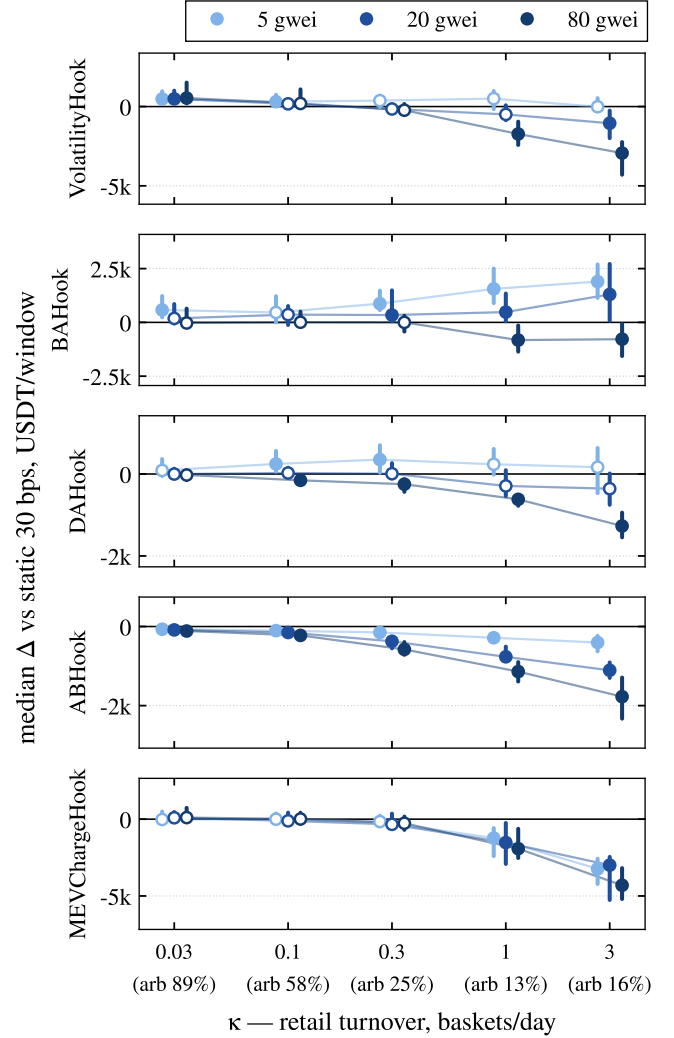


Fig. 4. Paired LP-outcome difference relative to static 30 bps across the five ordinary-user demand levels during high-volatility windows. Values above zero favor the hook. Colours distinguish the 5, 20, and 80 gwei gas-price scenarios. The percentages below the horizontal axis report the arbitrage share measured under the static 30 bps baseline.

changing the fee policy, following the principles of controlled AMM backtesting [22]. Deterministic mock feeds and local execution make these comparisons repeatable, but they do not reproduce live-oracle latency, aggregation, or failure behavior. Future work should add version-pinned execution environments, structured event-level output, stale-oracle fault injection, property-based contract tests, and public-testnet validation. The same framework can also support emerging mechanisms such as trade-size-aware and path-independent fee policies [23].

VI. CONCLUSION

This paper presented a reproducible framework for deploying and evaluating dynamic-fee hooks on Uniswap v4. The framework separates fee-policy logic from protocol setup and connects common Solidity implementations, permission-aware deployment, historical market observations, local EVM

execution, and comparative LP metrics. Using this common workflow, we compared five dynamic hooks with four static-fee configurations in 29,160 experimental cells spanning three token pairs, 72 one-day windows per pair, three gas-price scenarios, and five demand levels. No dynamic policy outperformed the predeclared 30 bps baseline unconditionally. Improvements were confined to high-volatility, low-gas conditions and depended on the balance between ordinary-user and arbitrage volume; higher gas costs eliminated these gains. The contribution is therefore both engineering and empirical: reusable infrastructure for controlled hook comparison and evidence that dynamic fees are conditionally, rather than universally, beneficial.

REFERENCES

- [1] Guillermo Angeris, Hsien-Tang Kao, Rei Chiang, Charlie Noyes, and Tarun Chitra. An analysis of uniswap markets, 2019.
- [2] Guillermo Angeris, Akshay Agrawal, Alex Evans, Tarun Chitra, and Stephen Boyd. Constant function market makers: Multi-asset trades via convex optimization. In *Handbook on Blockchain*, pages 415–444. Springer, 2022.
- [3] Jason Milionis, Ciamac C. Moallemi, Tim Roughgarden, and Anthony Lee Zhang. Automated market making and loss-versus-rebalancing, 2022.
- [4] Irina Lebedeva, Dmitrii Umnov, Yury Yanovich, Ignat Melnikov, and George Ovchinnikov. Dynamic fee for reducing impermanent loss in decentralized exchanges, 2025.
- [5] Leonardo Baggiani, Martin Herdegen, and Leandro Sánchez-Betancourt. Optimal dynamic fees in automated market makers, 2025.
- [6] Hayden Adams, Noah Zinsmeister, and Dan Robinson. Uniswap v2 core. White paper, 2020.
- [7] Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, and Dan Robinson. Uniswap v3 core. White paper, 2021.
- [8] Zhou Fan, Francisco Marmolejo-Cossio, Daniel J. Moroz, Michael Neuder, Rithvik Rao, and David C. Parkes. Strategic liquidity provision in uniswap v3, 2021.
- [9] Roman Vlasov, Vladimir Gorgadze, and Artem Barger. The Impact of the Exchange Fees on Impermanent Loss of Liquidity Providers for Conservative Automated Market Makers. *The Journal of The British Blockchain Association*, may 27 2025.
- [10] Robin Fritsch. A note on optimal fees for constant function market makers. In *Proceedings of the 2021 ACM CCS Workshop on Decentralized Finance and Security*, CCS '21, page 9–14. ACM, November 2021.
- [11] Steven Campbell, Philippe Bergault, Jason Milionis, and Marcel Nutz. Optimal fees for liquidity provision in automated market makers, 2025.
- [12] Daniele Maria Di Nosse and Fabrizio Lillo. Mitigating adverse selection in concentrated liquidity AMMs with dynamic fees: An agent-based model approach, 2026.
- [13] Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, and Dan Robinson. Uniswap v4 core. White paper, 2024.
- [14] Uniswap Labs. Dynamic fees. Uniswap v4 Developer Documentation, 2026. Accessed: 2026-07-17.
- [15] Guillermo Angeris and Tarun Chitra. Improved price oracles: Constant function market makers. In *Proc. 2nd ACM Conf. Advances in Financial Technologies*, pages 80–91, 2020.
- [16] Ignat Melnikov, Roman Vlasov, Vladimir Gorgadze, Andrey Seoev, and Yury Yanovich. From impermanent loss to sustainable gain: Quantifying profitability zones for liquidity providers on dex. In *IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, 2026, to appear.
- [17] Uniswap Labs. Hooks. Uniswap v4 Developer Documentation, 2026. Accessed: 2026-07-17.
- [18] Uniswap Labs. Hook deployment. Uniswap v4 Developer Documentation, 2026. Accessed: 2026-07-17.
- [19] OpenZeppelin. Uniswap hooks. Developer documentation, 2026. Accessed: 2026-07-17.
- [20] Chainlink Labs. Chainlink data feeds. Developer documentation, 2026. Accessed: 2026-07-17.
- [21] Uniswap Labs. Uniswap v4 security framework. Developer documentation, 2026. Accessed: 2026-07-17.
- [22] Andrey Urusov, Rostislav Berezovskiy, and Yury Yanovich. Backtesting framework for concentrated liquidity market makers on uniswap v3 decentralized exchange. *Blockchain: Research and Applications*, 6(1):100256, 2025.
- [23] Andrey Voronin, Roman Vlasov, Vladimir Gorgadze, Andrey Seoev, and Yury Yanovich. Characterizing path-independent fees: A route to zero impermanent loss in cpmms. In *2026 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pages 1–7, 2026.